🚀 SwiftURL — The Complete Project Guide

Rate-Limited URL Shortener (PERN Stack) | Engineered by Koushik Bobba

1. WHAT IT DOES: The Core Concept

SwiftURL takes long, messy web links (e.g., `https://amazon.com/category/laptop/...`) and converts them into short, clean, 6-character links (e.g., `http://localhost:8080/r/q0V`). When a user clicks the short link, they are instantly redirected to the original long URL. Simultaneously, the system tracks analytics—recording the user's IP, browser (User-Agent), and timestamp—without slowing down the redirect experience.

2. HOW IT WORKS: The End-to-End Pipeline

```
USER'S BROWSER (React UI)
        │
        ▼
EXPRESS.JS BACKEND (Port 8080)
        │
        ├──► REDIS          ← Rate Limits requests + Caches URLs for O(1) instant lookups
        │
        ├──► POSTGRESQL     ← Permanently stores Users and URL mappings
        │
        └──► RABBITMQ       ← Receives click events instantly (Async queue)
                │
                ▼
        NODE.JS CONSUMER WORKER
                │
                ▼
        POSTGRESQL          ← Safely writes click data to the database in the background
```

3. TECHNOLOGY STACK: What We Used & Why

Node.js & Express.js — Backend API

What: Handles HTTP routing, JSON web tokens, and the core logic of the application.

Why: The JavaScript ecosystem allows for rapid iteration and seamless code-sharing context with the React frontend. It provides excellent asynchronous I/O performance which is critical for handling high concurrent traffic.

PostgreSQL — Primary Database

What: A relational database storing `users`, `urls`, and `click_analytics`.

Why: We need strict data integrity. If a user is deleted, all their URLs should be deleted (`ON DELETE CASCADE`). PostgreSQL's MVCC (Multi-Version Concurrency Control) ensures that heavy writes (analytics) don't lock out heavy reads (redirects).

Redis Caching & Rate Limiting

What: An in-memory data structure store.

Why Cache: Database queries take ~10ms. Redis fetches from RAM in ~0.1ms. Popular short links are cached so PostgreSQL is never touched for a redirect (Cache-Aside pattern).

Why Rate Limiter: We use Redis Sorted Sets (`ZSET`) to build a Sliding-Window Rate Limiter. It tracks exactly how many requests an IP made in the last 60 seconds and blocks them if they exceed 10 requests, protecting us from DDoS attacks.

RabbitMQ Message Queue

What: A robust message broker for asynchronous communication.

Why: When a user clicks a link, writing analytics to PostgreSQL takes 20-50ms. If 1,000 users click at once, the database would freeze. Instead, the Express API sends a 302 redirect instantly (<2ms) and tosses a tiny message to RabbitMQ ("Someone clicked this!"). A separate worker reads this queue and writes to PostgreSQL at a safe pace.

React + Vite Frontend

What: The user-facing dashboard.

Why: We built a Single Page Application (SPA) with dynamic rendering. It uses `Recharts` for interactive SVG graphs, `lucide-react` for crisp icons, and custom CSS for a beautiful "Glassmorphic" (frosted glass) design. Vite is used over Webpack because its native ESM module loading is 10x faster for development.

4. KEY ENGINEERING ALGORITHMS

Base62 Encoding for URL Generation:

Instead of generating random short codes (which can cause collisions/duplicates), we use a PostgreSQL Sequence `nextval()` to get an atomic incrementing number (1, 2, 3...). We then convert that number into Base62 (0-9, a-z, A-Z). Since every number is unique, every short code is mathematically guaranteed to be unique. Zero collisions!

5. ISSUES FACED & HOW WE SOLVED THEM

Issue 1: The Docker Daemon Connection Failure

Problem: When trying to run `docker compose build`, the system threw an error: `failed to connect to the docker API at npipe:////./pipe/dockerDesktopLinuxEngine`.

Solution: This occurred because the Docker Desktop GUI/Engine wasn't actively running on Windows. The fix was manual intervention: starting the Docker Desktop application from the Windows Start menu to initialize the daemon before running CLI commands.

Issue 2: GitHub Actions CI Node.js Version Mismatch

Problem: Our initial CI pipeline failed to build the React frontend. The runner defaulted to Node 18, but Vite 8 requires Node >= 20.19.0 or 22.12.0.

Solution: We updated the `.github/workflows/ci.yml` file to explicitly configure `actions/setup-node@v3` with `node-version: 22`.

Issue 3: C++ to PERN Architecture Migration

Problem: The project originally utilized a C++ backend. However, it was harder to maintain and test, and integrating standard web libraries was complex.
Solution: We migrated the entire backend architecture to the PERN stack (PostgreSQL, Express, React, Node). We ripped out the C++ core and replaced it with Node.js/Express, while successfully reusing the *exact same* PostgreSQL database schema and RabbitMQ worker without needing to alter them.

6. TOP INTERVIEW QUESTIONS & ANSWERS

Q1: How did you implement Rate Limiting and why that method?

I used the **Sliding Window Log** algorithm via Redis Sorted Sets (`ZSET`). For every request, we log the timestamp as both the score and value. We then use `ZREMRANGEBYSCORE` to drop requests older than 60 seconds, and `ZCARD` to count remaining requests. This is superior to Fixed Window (which allows double-bursts at the minute boundary) because it provides true rolling-window precision.

Q2: Why use RabbitMQ? Why not just write to the database directly?

In a URL shortener, the read-to-write ratio is heavily skewed (thousands of redirects for every 1 URL created). If a link goes viral, hitting the database synchronously for every click would cause a connection pool bottleneck and crash the server. RabbitMQ decouples the process: the server returns a 302 redirect in under 2ms, drops a message in the queue, and the background worker updates the database when it has capacity.

Q3: How do you guarantee a short code is unique without querying the database every time?

By using Base62 encoding on a monotonically increasing counter (PostgreSQL sequence). Random strings suffer from the Birthday Paradox, requiring expensive collision checks. A counter ensures that ID 1 becomes '1', ID 2 becomes '2', ID 100000 becomes 'q0V'. Every input is unique, so every output is unique by definition.

Q4: What is Cache-Aside and how did you use it?

Cache-Aside is a caching strategy where the application checks the cache first. If there's a "hit", it returns the data immediately. If there's a "miss", the app queries the primary database, stores the result in the cache, and then returns it. In this project, Redis caches the `short_code -> original_url` mapping, ensuring 99% of redirects never touch PostgreSQL.

Q5: Explain how you secured the application.

1. Passwords are hashed using `bcrypt` with a salt round of 10.
2. Authentication is stateless using JSON Web Tokens (JWT), preventing session hijacking.
3. Redis rate limiting protects the API from DDoS and brute-force attacks.
4. Prepared SQL statements via the `pg` library prevent SQL Injection attacks.